

Bilkent University, CS 473/573, Session 2, 24pts, 30min**Q2:**

- a. An algorithm solves a problem of size n by dividing it into eight subproblems, each of size $n/3$ in $\Theta(n^2)$ time, and combining the solutions of the subproblems in $\Theta(n)$ time. Find the running time of this algorithm in asymptotic notation.
- b. Find a **tight** asymptotic upper bound for the running time of an algorithm with the provided recurrence using the **Substitution** method:
- $$T(n) = 8T(n/2) + n^2, \quad n > 1$$
- $$T(1) = O(1)$$

Substitution Method:

1. Guess the asymptotic complexity.
2. Prove your guess using induction
 - (a) Assume inductive hypothesis holds for $k < n$
 - (b) Try to prove the general case for n

Note: You MUST prove the EXACT inequality, you CANNOT ignore lower order terms. If the proof fails, you could strengthen the inductive hypothesis and try again.

Master Method:

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$, $a \geq 1$, $b > 1$, and f is asymptotically positive, then:

- 1: $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$
- 2: $f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0 \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$, $\epsilon > 0$, and $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$, for some $c < 1 \Rightarrow T(n) = \Theta(f(n))$

You may use the following algorithms and the like covered in the lectures/book as indicated below:

- *INSERTION-SORT*(A, n): Insertion sort on $A[1 \dots n]$
- *MERGE-SORT*(A, p, r): Merge sort on $A[p \dots r]$
- *BINARY-SEARCH*(A, p, r, key): Binary search for key on $A[p \dots r]$
- *H-PARTITION*(A, p, r): Hoare's partitioning on $A[p \dots r]$, returns the last position of the region $\leq pivot$
- *L-PARTITION*(A, p, r): Lomuto's partitioning on $A[p \dots r]$, returns the last position of the region $\leq pivot$
- *QUICKSORT*(A, p, r): Quicksort on $A[p \dots r]$
- *R-QUICKSORT*(A, p, r): Randomized quicksort on $A[p \dots r]$
- *R-H-PARTITION*(A, p, r): Randomized Hoare's partitioning on $A[p \dots r]$ with a randomly selected pivot, returns the last position of the region $\leq pivot$
- *R-L-PARTITION*(A, p, r): Randomized Lomuto's partitioning on $A[p \dots r]$ with a randomly selected pivot, returns the last position of the region $\leq pivot$
- *R-SELECT*(A, p, r, i): Randomized select algorithm on array $A[p \dots r]$, returns the value of the i th smallest element
- *MM-SELECT*(S, n, i): Median of medians algorithm on set S containing n elements, returns the value of the i th smallest element
- *HEAP-EXTRACT-MAX*(A, n): Extract max on the heap of n elements stored in $A[1 \dots n]$
- *MAX-HEAPIFY*(A, i, n): Max heapify on the i th element of the heap stored in $A[1 \dots n]$
- *BUILD-MAX-HEAP*(A, n): Builds heap in $A[1 \dots n]$ using the n elements given in $A[1 \dots n]$
- *HEAPSORT*(A, n): Heapsort on $A[1 \dots n]$
- *MAX-HEAP-INSERT*(A, key, n): Inserts key into the heap stored in $A[1 \dots n]$
- *HEAP-INCREASE-KEY*(A, i, key): Increases the value of the i th element of the heap to key
- *COUNTSORT*(A, n, k): Applies counting sort on $A[1 \dots n]$ of integers in range $[1 \dots k]$